

Problema A

Na última semana, vimos tabelas Hash em que cada endereço-base da tabela Hash é o início de uma lista encadeada. Quando uma chave é mapeada para um endereço que já está ocupado (colisão), o novo elemento é simplesmente adicionado ao final da lista encadeada correspondente àquele endereço. A função Hash utilizada foi:

$$h(x) = x \% M, \text{ onde } x \text{ é a chave e } M \text{ é o número de endereços-base.}$$

Dessa vez, você deverá desenvolver um programa que simule a inserção de um conjunto de chaves em tabelas Hash usando a mesma função de dispersão, mas com tratamento de colisão por meio de endereçamento aberto.

Em tratamento de colisões por endereçamento aberto, quando ocorre colisão a chave a ser inserida na tabela é inserida no próximo endereço vazio na tabela (considerando o endereçamento circular), seguindo a seguinte função:

$$h(x) = (x+i)\%M, \text{ para } i=1,2,\dots$$

Entrada

A entrada contém vários casos de teste. A primeira linha da entrada contém um inteiro N , a quantidade de casos de teste. Cada caso de teste consiste em duas linhas:

1. A primeira linha contém dois inteiros: M , o número de endereços base (tamanho da tabela Hash) e C ($C \leq M$), a quantidade de chaves a serem inseridas na tabela.
2. A segunda linha contém C valores, separados por um espaço, os valores a serem inseridos na tabela Hash.

Saída

Para cada caso de teste, o programa deverá imprimir M linhas com 2 inteiros, o índice da chave na tabela Hash e a chave associada a ele.

Uma linha em branco deve ser impressa para separar os resultados de casos de teste diferentes.

OBS.: Para os endereços da tabela em que não houver nenhuma chave associada, deverá ser impresso o valor 0. É garantido que não haverá nenhum 0 como valor de entrada a ser inserido na tabela.

Exemplo de entrada	Exemplo de saída
2	0 0
13 9	1 27
44 45 49 70 27 73 92 97 95	2 92
7 8	3 0
35 12 2 17 19 51 88 86	4 95

	5 44
	6 45
	7 70
	8 73
	9 97
	10 49
	11 0
	12 0
	0 35
	1 88
	2 2
	3 17
	4 51
	5 12
	6 19

Você pode utilizar a seguinte estrutura para desenvolver o seu programa:

```
#include <stdio.h>
#include <stdlib.h>
```

```
typedef struct TabelaHash {
    int *tabela;
    int tamanho;
} TabelaHash;
```

```
int funcaoHash(int chave, int tamanho) {
    return abs(chave) % tamanho;
}
```

```
TabelaHash *criaTabelaHash(int tamanho){
    TabelaHash *th = (TabelaHash*) malloc (sizeof(TabelaHash));

    // comando calloc já aloca um vetor preenchendo todos os valores com
    0
    th->tabela = (int*) calloc (tamanho, sizeof(int));
    th->tamanho = tamanho;
    return th;
}
```

```
int main(){
    int n;
    scanf("%d", &n);x
```

```
for(int i = 0; i < n; ++i){
    int tamanho, c;
    scanf("%d %d", &tamanho, &c);
    TabelaHash *tabela_hash = criaTabelaHash(tamanho);

    /* COMPLETAR */
}

return 0;
}
```